ACS 560: Software Engineering

# Final Project Documentation

## SmartSpend: Personal Expense Tracker


**Team Members**

Shreya Sagar (Team Leader)

Shreya Komarabattini

Khushi Patel

Pushan Verma


Methodology: Incremental (Agile) | Version: 1.0

## Team Member Contributions

| Team Member | Component 1: Requirements | Component 2: Design | Component 3: Tools | Other |
|---|---|---|---|---|
| **Shreya Sagar** | Backend Auth Setup (User Model, Register/Login APIs) | JWT Authentication, Middleware Design | Node.js, Express, JWT, bcrypt, Postman | MongoDB connection, server setup |
| **Khushi Patel** | UI/UX Wireframes, Project Setup | Frontend Auth Design (Login/Register, Routing) | React, Redux Toolkit, Axios | Initial project structure setup (frontend + backend) |
| **Shreya Komarabattini** | Transaction Requirements (CRUD, Validation) | API + Component Design (Transaction Flow) | MongoDB, Express, React | Integration of frontend with backend transactions |
| **Pushan Verma** | Performance, Filters, Final Features | Dashboard Design, Charts, UI/UX Polish | React, Chart.js, Jest, Supertest | Testing, bug fixing, final integration |

# Part 1: Introduction, Purpose, and Goals

## 1.1 Introduction

SmartSpend is a comprehensive web-based personal expense tracking application designed for students and young professionals. It addresses the challenge of financial management by providing a centralized platform for monitoring income and expenses, enabling users to make informed financial decisions and develop healthy financial habits.

The system uses the MERN (MongoDB, Express, React, Node.js) technology stack and features secure user authentication, real-time transaction management, interactive data visualization, and comprehensive filtering capabilities.

## 1.2 Problem Statement

Students and young professionals face several financial management challenges:

- Lack of Visibility: Users cannot easily see where their money is being spent.
- Manual Tracking Issues: Spreadsheets and handwritten notes are time-consuming and error-prone.
- Absence of Centralized System: No single, easy-to-access location for all financial data.
- Limited Analysis Capabilities: Difficulty identifying spending patterns and trends.
- Complexity of Existing Solutions: Current finance apps are often too complex or overloaded with unnecessary features.

## 1.3 Purpose and Goals

Primary Purpose: To provide a simple, secure, and student-friendly expense tracking platform that enables users to:

- Track income and expenses in real-time
- Categorize and analyze spending patterns
- Visualize financial data through interactive charts
- Make data-driven financial decisions

**Project Goals:**

- User-Centric Design: Create an intuitive interface with minimal learning curve
- Financial Visibility: Provide clear insights into spending habits
- Performance: Ensure fast response times (dashboard load under 2 seconds)
- Accessibility: Support all device types through responsive design
- Scalability: Handle growing users and transaction volume

**Success Metrics:**

- User onboarding within 2 minutes

- Transaction creation under 15 seconds
- Dashboard loads under 2 seconds
- Zero security breaches

## 1.4 Project Constraints

**Technical Constraints:**

- MERN stack only (MongoDB, Express, React, Node.js)
- Must support modern browsers: Chrome, Firefox, Safari, Edge
- Requires internet connection for cloud-based database
- MongoDB as the primary database engine

**Operational Constraints:**

- 8-week development timeline (February 3 to March 30, 2026)
- 4-member development team
- Only free and open-source tools permitted
- Team members required to learn new technologies during development

**Design Constraints:**

- Responsive design supporting mobile, tablet, and desktop
- React Single Page Application (SPA) architecture
- RESTful API architecture
- JWT authentication and bcrypt password hashing

**Business Constraints:**

- Target audience: students and young professionals
- Free application with no subscription model
- Strong privacy requirements for financial data
- Basic data protection compliance

## Part 2: Project Glossary

The following terms and definitions are used throughout the SmartSpend project documentation.

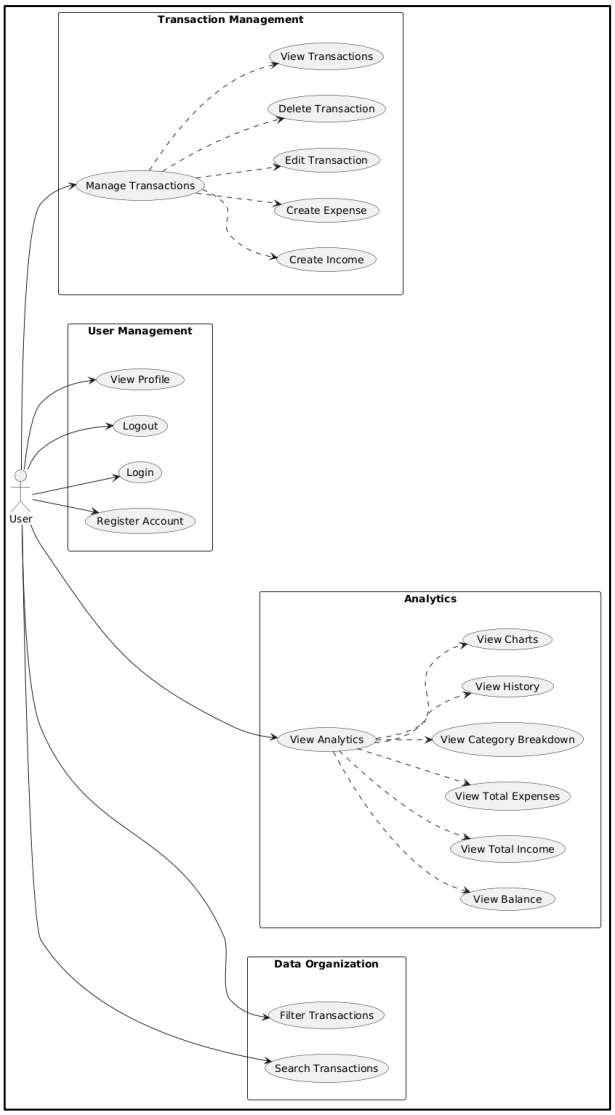
| Term | Definition |
|---|---|
| **API** | A set of rules enabling communication between frontend and backend services via HTTP. |
| **Authentication** | The process of verifying user identity using JWT tokens before granting access. |
| **Backend** | Server-side logic built with Node.js and Express that processes business rules and data. |
| **Balance** | Net financial position calculated as Total Income minus Total Expenses. |
| **Bcrypt** | Industry-standard password hashing algorithm using salt rounds to resist brute-force attacks. |
| **Category** | A label classifying a transaction (e.g., Food, Rent, Travel, Other). |
| **CRUD Operations** | The four fundamental database operations: Create, Read, Update, Delete. |
| **Dashboard** | The main screen displaying financial summaries, transaction management, and analytics. |
| **Expense** | A monetary outflow transaction that reduces the user's balance. |
| **Filter** | A feature narrowing the displayed transactions by type, category, or date range. |
| **Frontend** | The client-side interface built with React.js that the user interacts with. |
| **Income** | A monetary inflow transaction that increases the user's balance. |
| **JWT** | JSON Web Token — compact token for stateless authentication; expires after 7 days. |
| **MERN Stack** | MongoDB, Express.js, React.js, Node.js — the full-stack JavaScript framework used. |
| **MongoDB** | A NoSQL document-oriented database storing user and transaction data. |
| **Mongoose** | An ODM library for MongoDB providing schema validation and query building. |
| **Pagination** | Splitting large datasets into pages; SmartSpend displays 50 transactions per page. |
| **Protected Route** | An API endpoint or page requiring a valid JWT token to access. |
| **REST** | Representational State Transfer — API design using standard HTTP methods. |
| **Schema** | Structural definition of a database document, specifying fields, types, and validation. |
| **Session** | An active authenticated period managed by storing a JWT in localStorage. |
| **Transaction** | A single financial record (income or expense) with amount, category, and date. |

| | |
|---|---|
| **User** | A registered account holder with personal, isolated financial data. |
| **Validation** | Checking user input for correctness and completeness before processing. |
| **Visualization** | Charts and graphs rendered via Chart.js to provide spending insights. |

# Part 3: Use Case and Class Models

## 3.1 Use Case Diagram

The use case diagram illustrates all interactions between the primary actor (User) and the SmartSpend system. The diagram covers all 20 functional requirements identified in the SRS document.



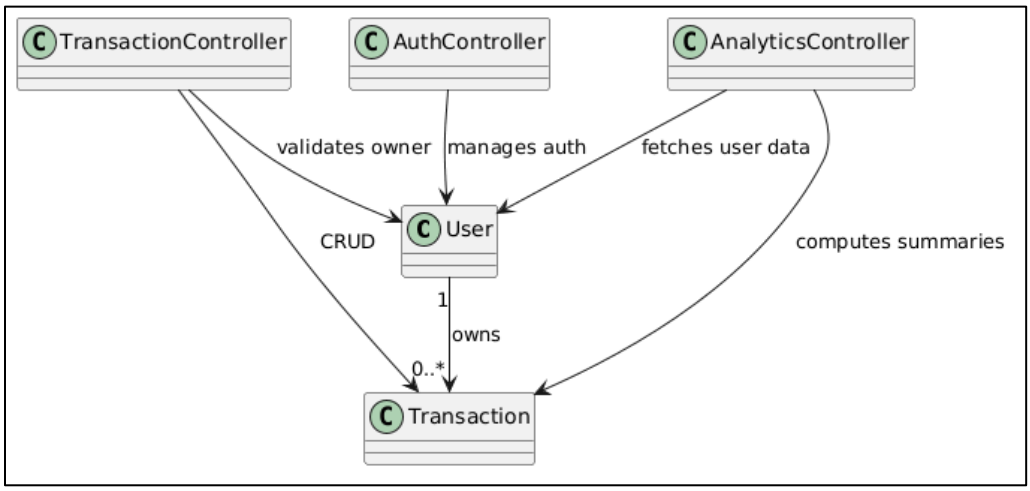
Use cases are organized into four groups:

- User Management: Register Account, Login, Logout, View Profile
- Transaction Management: Create Income, Create Expense, Edit Transaction, Delete Transaction, View Transactions
- Data Organization: Categorize Transactions, Filter by Type, Filter by Category, Filter by Date, Search Transactions
- Analytics and Visualization: View Balance, View Total Income, View Total Expenses, View Category Breakdown, View History, View Charts

The use case diagram illustrates the interactions between the primary actor (User) and the SmartSpend system. It captures all major functionalities including user management, transaction management, data organization, and analytics. Each use case represents a specific feature that allows the user to track, manage, and analyze financial data efficiently.

### 3.2 Class Diagram

The class diagram shows the two primary domain classes (User and Transaction) and their relationships, without detailing individual attributes or methods.




**Core Classes:**

- User: Represents a registered account holder with authentication credentials
- Transaction: Represents a single financial record owned by a User (one-to-many relationship)
- AuthController: Handles registration and login business logic
- TransactionController: Handles CRUD operations for transactions
- AnalyticsController: Computes balance summaries and category breakdowns

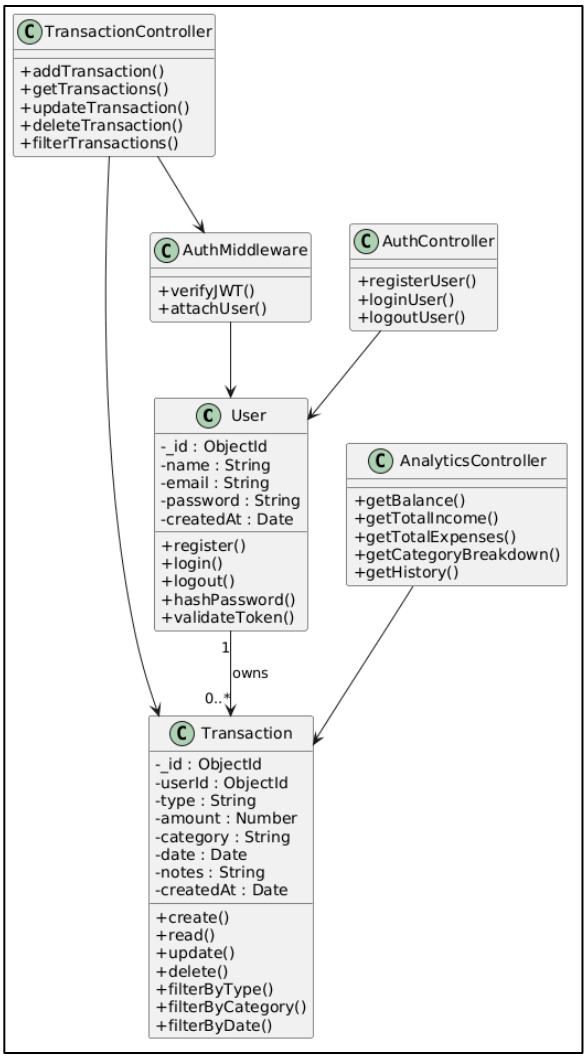
The class diagram represents the high-level structure of the system by identifying the core entities and their relationships. The main classes include User and Transaction, where a one-to-many relationship exists between them. Controller classes such as AuthController, TransactionController, and AnalyticsController manage the business logic and interactions between the user interface and the database.

## Part 4: Design and Implementation Modules

This section describes the key design models used to implement the SmartSpend system. Each model corresponds to a significant use case or architectural decision.

### 4.1 Class Model

The class model defines the structural blueprint of the SmartSpend system with attributes, methods, and relationships between primary classes.



User Class attributes: _id, name, email, password (hashed), createdAt. Methods: register(), login(), logout(), hashPassword(), validateToken().

Transaction Class attributes: _id, userId (ref), type (income/expense), amount, category, date, notes, createdAt. Methods: create(), read(), update(), delete(), filterByType(), filterByCategory(), filterByDate().

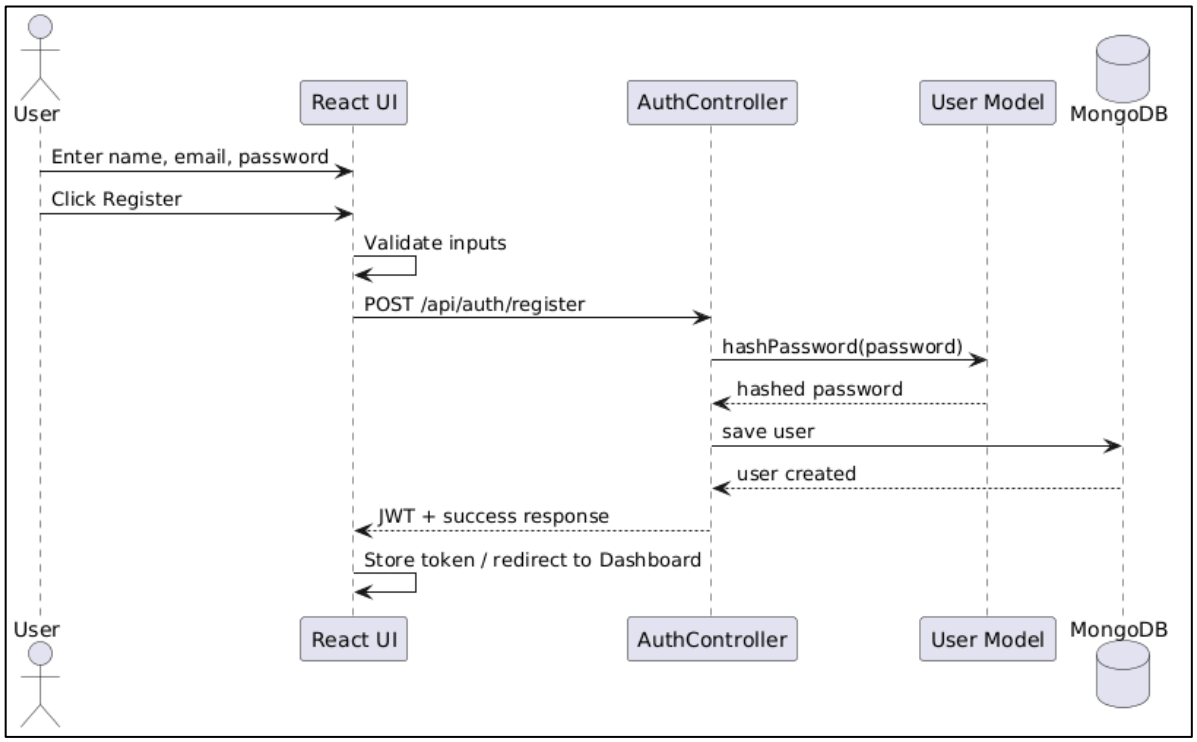
The User to Transaction relationship is one-to-many. Each transaction stores a userId foreign key referencing its owner.

The class model provides a detailed view of the system by defining the attributes, methods, and relationships between classes. It shows how user data and transaction data are structured and processed within the system. The model also highlights how controllers handle authentication, transaction operations, and analytics computation while maintaining separation of concerns.

## 4.2 Use Case Sequence Model

### 4.2.1 User Registration Sequence

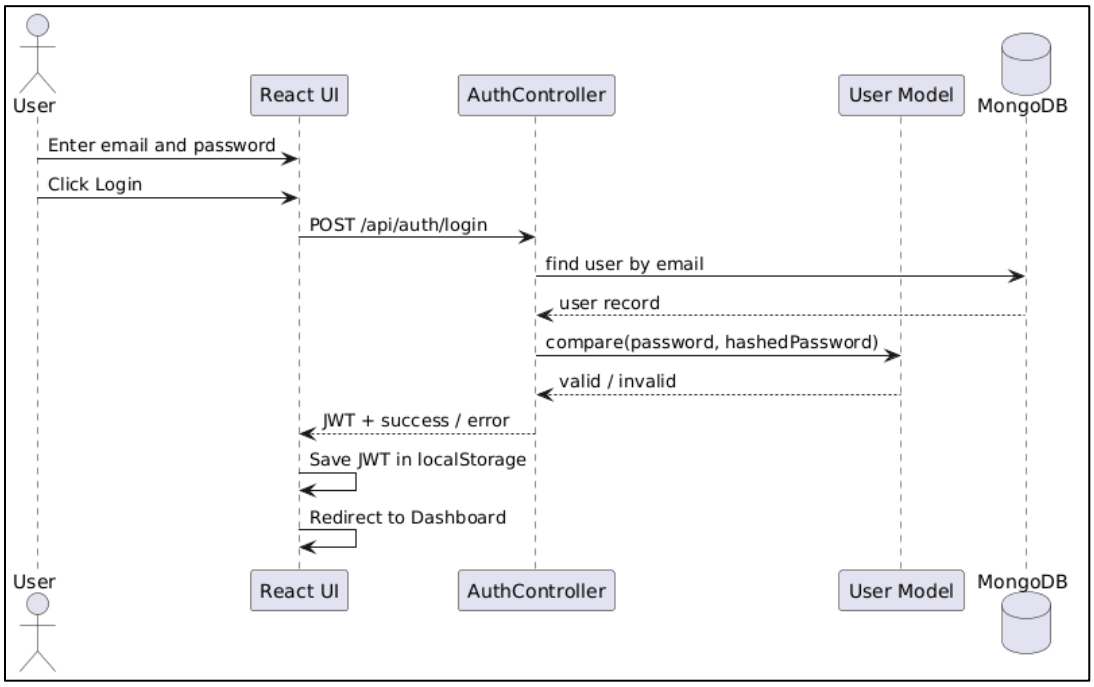
Shows the message flow when a new user registers: client-side validation, POST to /api/auth/register, password hashing via bcrypt, MongoDB storage, JWT response, and redirect to Dashboard.




This sequence diagram illustrates the flow of events during user registration. It begins with client-side validation, followed by a request to the backend API. The server processes the request by hashing the password and storing user data in the database, then returns a response with a JWT token. The user is then redirected to the dashboard upon successful registration.

### 4.2.2 User Login Sequence

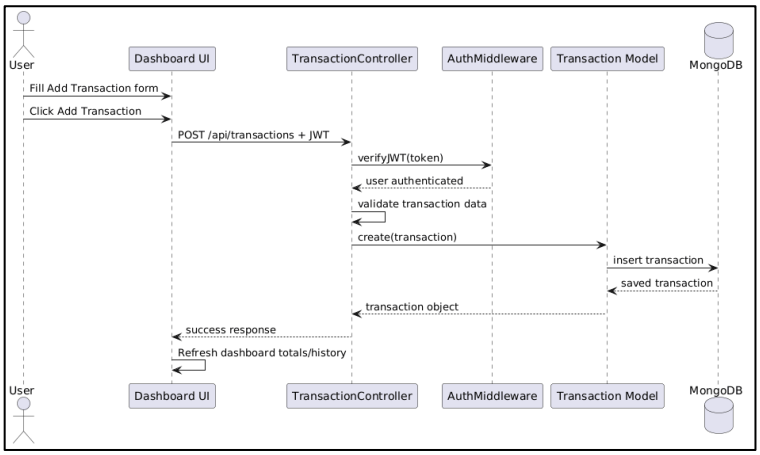
Shows the authentication flow: credential submission, POST to /api/auth/login, bcrypt.compare() validation, JWT generation, localStorage storage, and redirect to Dashboard.




This diagram shows the authentication process when a user logs into the system. The user submits credentials, which are validated by the backend using bcrypt. Upon successful verification, a JWT token is generated and sent to the client, allowing access to protected routes and features within the application.

### 4.2.3 Add Transaction Sequence

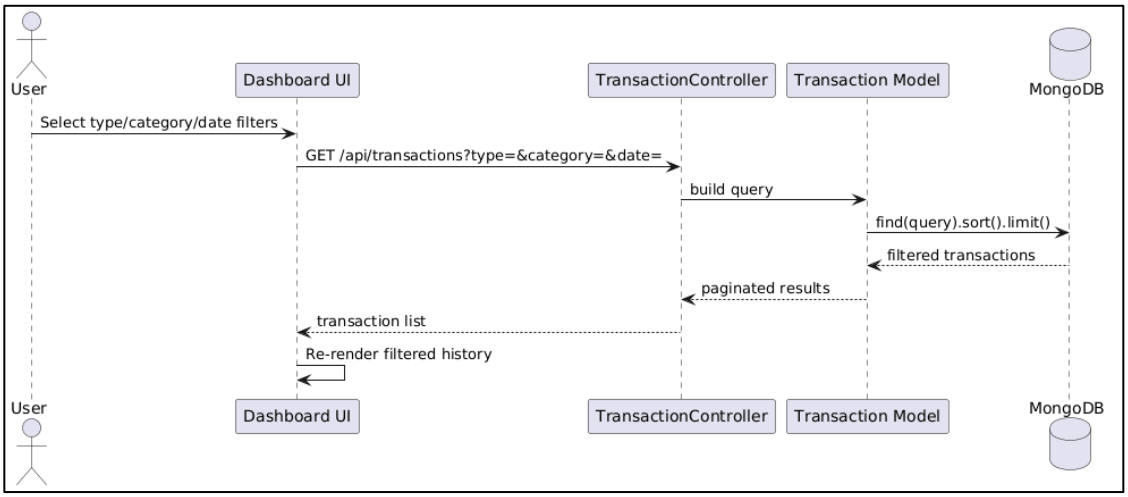
Illustrates adding a new transaction: form submission, authenticated POST to /api/transactions, authMiddleware JWT check, validation, MongoDB write, and real-time dashboard update.



This sequence diagram represents the process of adding a new transaction. The user submits transaction details through the frontend, which sends an authenticated request to the backend. The system validates the input, stores the transaction in the database, and updates the dashboard in real time to reflect the changes.

### 4.2.4 Filter Transactions Sequence

Shows how filtering works: user selects filter options, GET /api/transactions with query parameters, indexed MongoDB query, paginated response, and transaction list re-render.

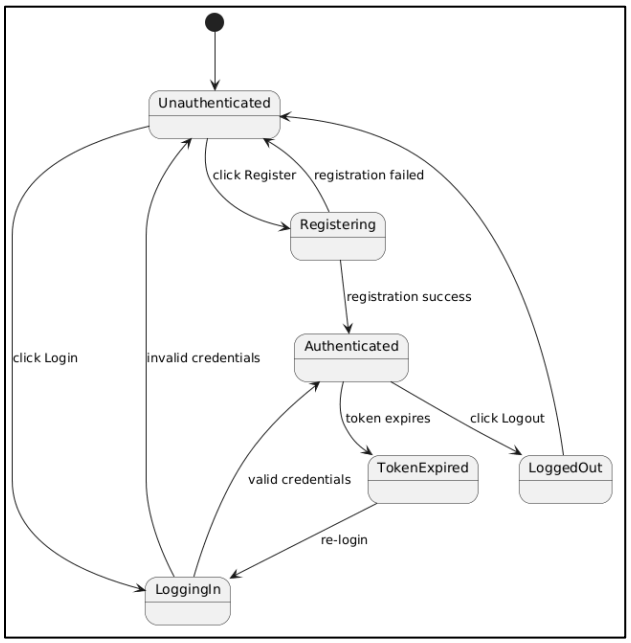



This diagram illustrates how transaction filtering is performed. The user selects filter criteria, which are sent as query parameters to the backend API. The database processes the query and returns filtered results, which are then displayed on the dashboard.

## 4.3 State Model

### 4.3.1 User Authentication State Machine

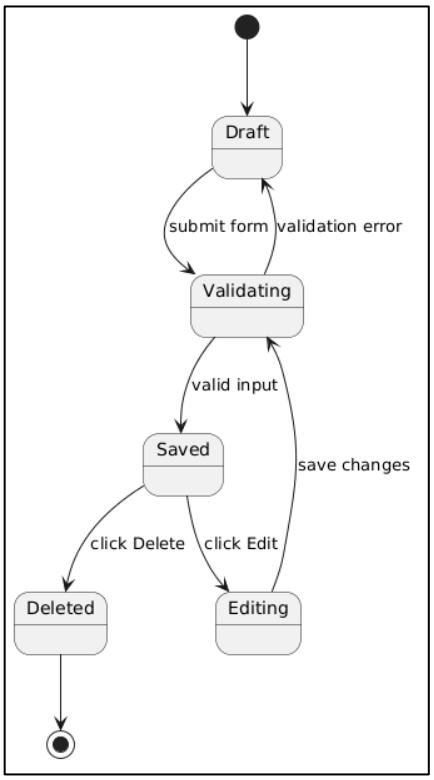
Describes the user session lifecycle: Unauthenticated, Registering, Logging In, Authenticated, Token Expired, and Logged Out states with transitions.




The state diagram models the lifecycle of user authentication within the system. It shows transitions between states such as Unauthenticated, Registering, Logging In, Authenticated, Token Expired, and Logged Out. These transitions represent how the system manages user sessions and access control.

### 4.3.2 Transaction State Machine

Describes the transaction lifecycle: Draft (form being filled), Validating, Saved, Editing, and Deleted states with transitions.

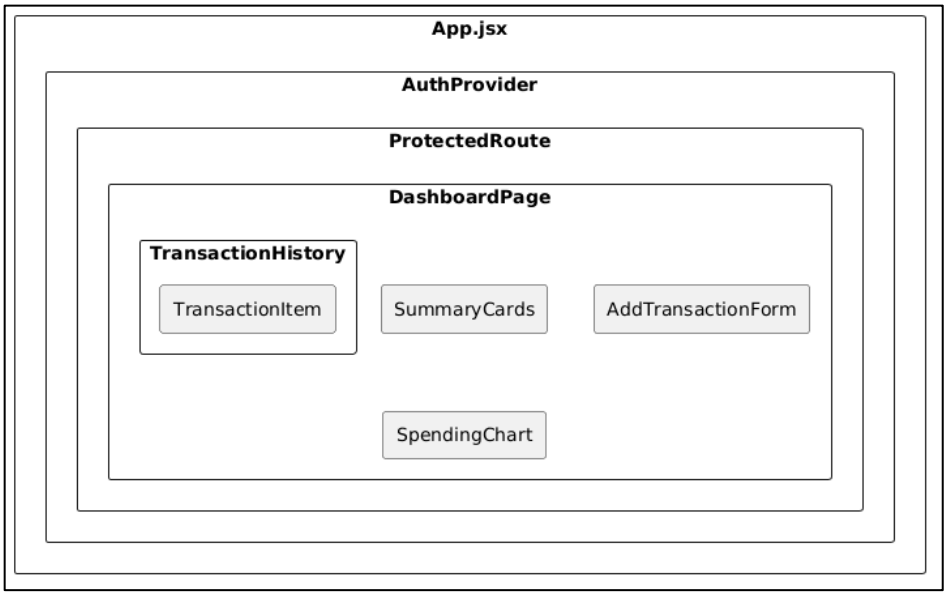


This diagram represents the lifecycle of a transaction from creation to deletion. It includes states such as Draft, Validating, Saved, Editing, and Deleted. The transitions show how transactions are created, modified, validated, and removed from the system.

### 4.4 Component Architecture

The React component hierarchy promotes reusability and separation of concerns:

- App.jsx: Root component. Wraps the app in Redux store and sets up client-side routing.
- AuthProvider: Redux-connected context managing global authentication state.
- ProtectedRoute: Higher-order component redirecting unauthenticated users to Login.
- DashboardPage: Main authenticated view orchestrating all sub-components.
- SummaryCards: Displays Total Balance, Total Income, and Total Expenses KPI cards.
- AddTransactionForm: Controlled form with real-time validation for adding transactions.
- SpendingChart: Chart.js doughnut chart showing category-wise spending distribution.
- TransactionHistory: Paginated, filterable list with inline Edit and Delete controls.
- TransactionItem: Single row component with category badge, amount, and action buttons.

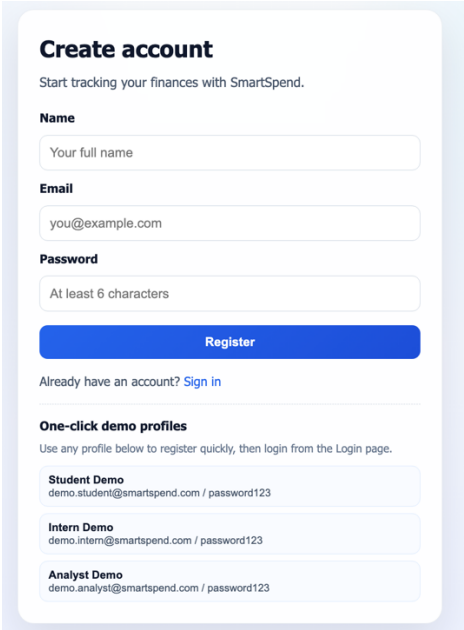



The component architecture diagram illustrates the structure of the frontend application and how different components interact. It highlights the separation of concerns by organizing the system into reusable components such as authentication, dashboard, transaction management, and visualization. This modular design improves maintainability and scalability.

## Part 5: Demo Screenshots

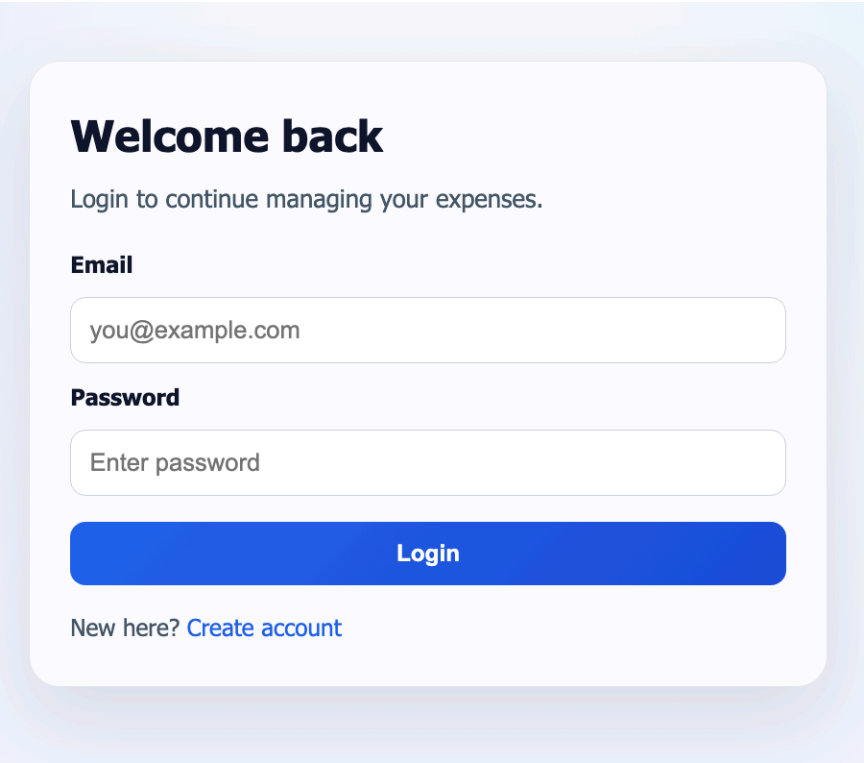
This section presents key screenshots demonstrating the vital data processing workflows in SmartSpend.

### 5.1 Registration Page



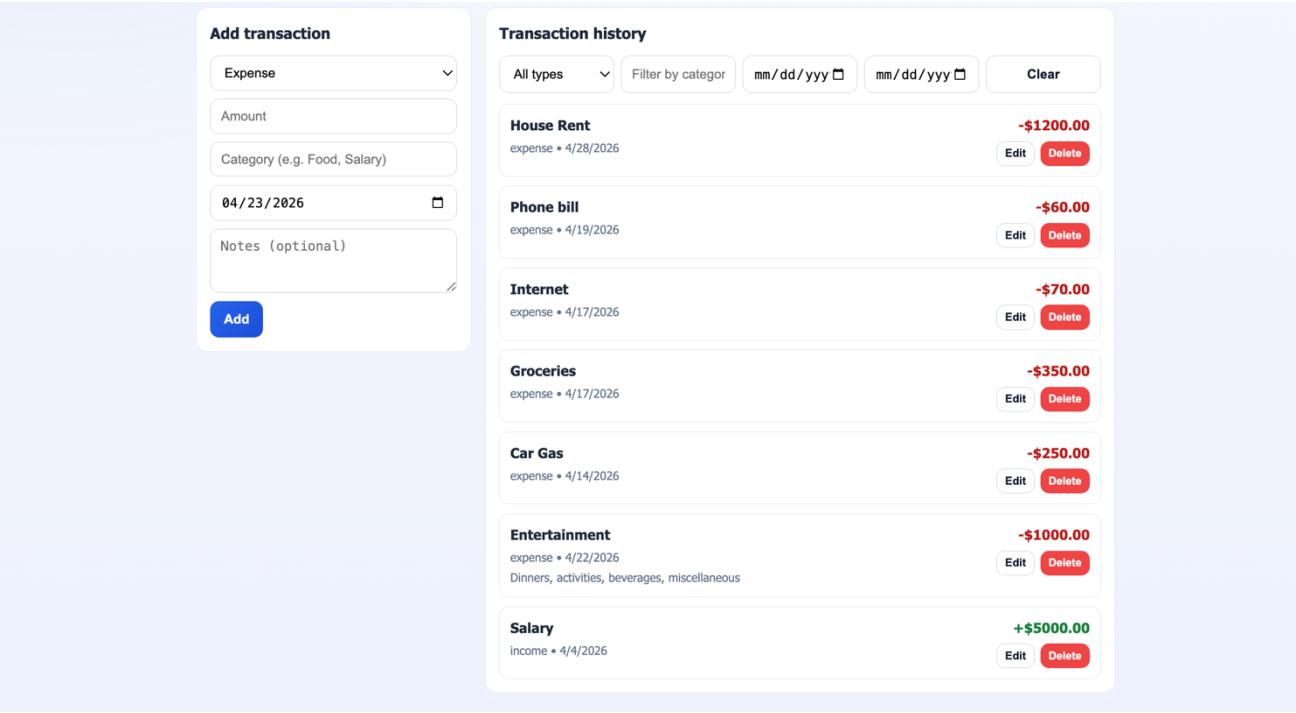

The Registration Page collects user details and validates inputs client-side before sending a POST /api/auth/register request.

### 5.2 Login Page



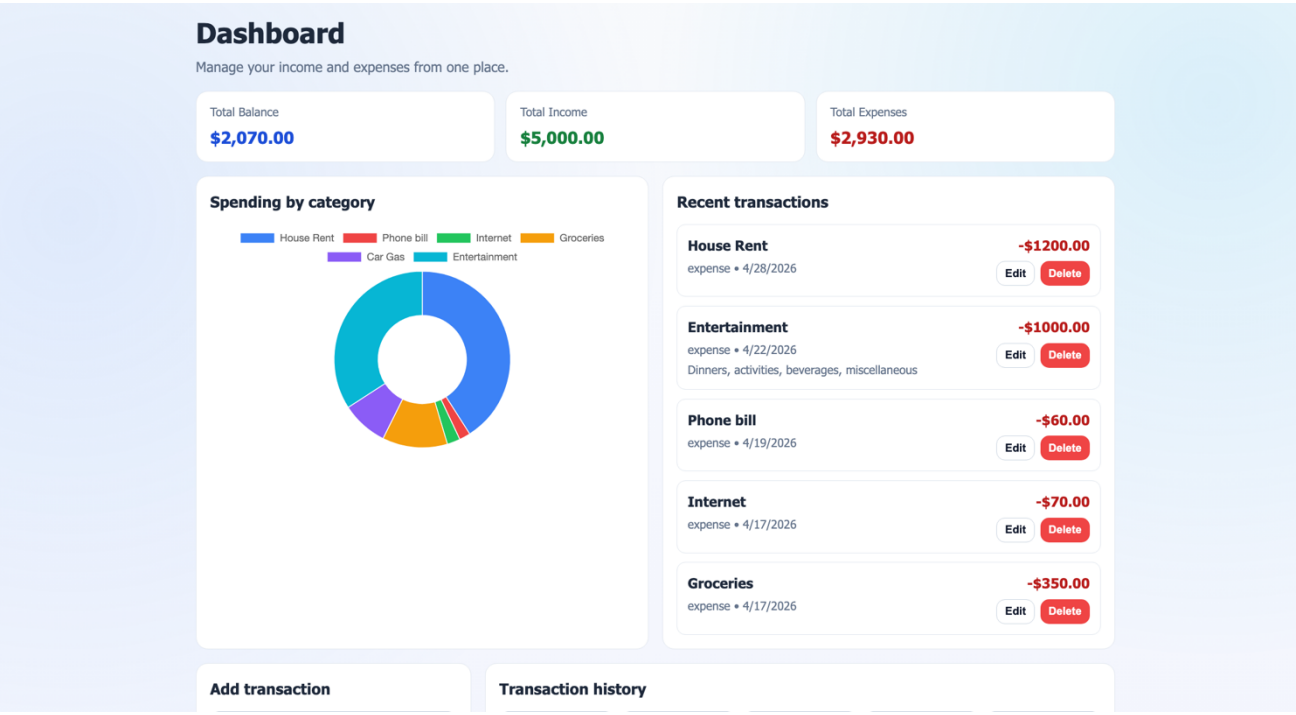

The Login Page authenticates users via POST /api/auth/login. The JWT is stored in localStorage and the user is redirected to the Dashboard.

## 5.3 Dashboard — Summary and Add Transaction




The Dashboard fetches all transactions via GET /api/transactions, computes balance and totals in real-time, and renders summary cards and the spending chart.

### 5.4 Recent Transaction with Filters and Spending Chart




Transactions are fetched, filtered by active state, sorted reverse-chronologically, and paginated at 50 items per page.

The chart visualizes the GET /api/analytics response using Chart.js, with data aggregated server-side by category.

## Part 6: Testing Report (Use Case)

This section provides formal test cases for the five major use cases of SmartSpend. Five test cases are developed per use case following the template specified in the assignment rubric. Actual Results and Status columns are to be completed upon test execution.

### 6.1 Use Case: User Registration

| Test Case Id | Test case description | Precondition | Test Data | Test Steps | Expected Results | Actual Results | Status (Pass/Fail) |
|---|---|---|---|---|---|---|---|
| Reg 1 | Successful registration with valid inputs | No existing account for the email. App is running. | Name: 'John Doe', Email: 'john@test.com', Password: 'pass123' | 1. Navigate to /register<br>2. Enter Full Name: 'John Doe' | User is registered and redirected to login page. | User was successfully registered and redirected to the login | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | 3. Enter Email: 'john@test.com'<br>4. Enter Password: 'pass123'<br>5. Click 'Create Account' | Account stored in MongoDB with hashed password. | page. Account confirmed in MongoDB with bcrypt-hashed password. | |
| Reg 2 | Registration with an already registered email | Account for 'john@test.com' already exists. | Name: 'Jane', Email: 'john@test.com', Password: 'newpass123' | 1. Navigate to /register<br>2. Enter Name: 'Jane'<br>3. Enter Email: 'john@test.com'<br>4. Enter Password: 'newpass123'<br>5. Click 'Create Account' | Error message displayed: 'Email is already registered.' Form is not submitted. | Error message 'Email is already registered.' appeared below the email field. Form was not submitted and no duplicate account was created. | Pass |
| Reg 3 | Registration with invalid email format | No pre-condition needed. | Name: 'Alice', Email: 'not-an-email', Password: 'pass123' | 1. Navigate to /register<br>2. Enter Name: 'Alice'<br>3. Enter Email: 'not-an-email'<br>4. Enter Password: 'pass123'<br>5. Click 'Create Account' | Validation error: 'Please enter a valid email address.' Form not submitted. | Inline validation error 'Please enter a valid email address.' displayed below the email field on submit. Form was not sent to the server. | Pass |
| Reg 4 | Registration with password below minimum length | No pre-condition needed. | Name: 'Bob', Email: 'bob@test.com', Password: 'abc' | 1. Navigate to /register<br>2. Enter Name: 'Bob'<br>3. Enter Email: 'bob@test.com' | Validation error: 'Password must be at least 6 characters.' | Validation error 'Password must be at least 6 characters.' displayed | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | 4. Enter Password: 'abc' (3 chars)<br>5. Click 'Create Account' | Form not submitted. | below the password field. Form submission was blocked. | |
| Reg 5 | Registration with all fields left blank | No pre-condition needed. | Name: '', Email: '', Password: '' | 1. Navigate to /register<br>2. Leave all fields empty<br>3. Click 'Create Account' | Validation errors shown for all three fields. Form not submitted. Required field indicators visible. | Validation errors appeared beneath all three fields. Form was not submitted and no API call was made. | Pass |

## 6.2 Use Case: User Login

| **Test Case Id** | **Test case description** | **Precondition** | **Test Data** | **Test Steps** | **Expected Results** | **Actual Results** | **Status (Pass/Fail)** |
|---|---|---|---|---|---|---|---|
| Login 1 | Successful login with valid credentials | User account exists: 'john@test.com', Password: 'pass123'. | Email: 'john@test.com', Password: 'pass123' | 1. Navigate to /login<br>2. Enter Email: 'john@test.com'<br>3. Enter Password: 'pass123'<br>4. Click 'Sign In' | User is authenticated. JWT stored in localStorage. Redirected to Dashboard. | User was authenticated successfully. JWT token confirmed in localStorage. Dashboard loaded with user data. | Pass |
| Login 2 | Login with incorrect password | Account exists for 'john@test.com'. | Email: 'john@test.com', Password: 'wrongpass' | 1. Navigate to /login<br>2. Enter Email: 'john@test.com'<br>3. Enter Password: 'wrongpass'<br>4. Click 'Sign In' | Error message: 'Invalid credentials. Please try again.' No token issued. | Error message 'Invalid credentials. Please try again.' was displayed. No JWT was issued and user remained on | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | the login page. | |
| Login 3 | Login with unregistered email | No account exists for the given email. | Email: 'ghost@no.com', Password: 'anypass' | 1. Navigate to /login<br>2. Enter Email: 'ghost@no.com'<br>3. Enter Password: 'anypass'<br>4. Click 'Sign In' | Error message: 'Invalid credentials. Please try again.' Same error as wrong password to prevent enumeration. | Same generic error 'Invalid credentials. Please try again.' was shown. Email existence was not revealed in the response. | Pass |
| Login 4 | Redirect to Dashboard if already logged in | User is already authenticated (valid JWT in localStorage). | Existing JWT token in localStorage. | 1. Open browser with valid JWT present<br>2. Navigate to /login | System detects existing valid token and auto-redirects to Dashboard without showing login form. | App detected the existing valid JWT and immediately redirected to the Dashboard. The login form was never rendered. | Pass |
| Login 5 | Login with empty fields | No pre-condition needed. | Email: '', Password: '' | 1. Navigate to /login<br>2. Leave both fields empty<br>3. Click 'Sign In' | Validation errors shown: 'Email is required.' and 'Password is required.' Sign In button is disabled. | Validation errors 'Email is required.' and 'Password is required.' appeared under each field. The Sign In button was disabled and no request was sent. | Pass |

## 6.3 Use Case: Add Transaction

| Test Case Id | Test case description | Precondition | Test Data | Test Steps | Expected Results | Actual Results | Status (Pass/Fail) |
|---|---|---|---|---|---|---|---|
| Add 1 | Add a valid expense transaction | User is logged in and on Dashboard. | Type: Expense, Amount: 45.99, Category: Food, Date: 2024-02-02, Notes: 'Lunch at restaurant' | 1. Select 'Expense' in Add Transaction form<br>2. Enter Amount: 45.99<br>3. Select Category: Food<br>4. Enter Date: 2024-02-02<br>5. Enter Notes: 'Lunch at restaurant'<br>6. Click 'Add Transaction' | Transaction saved. Total Expenses increases by $45.99. Balance decreases by $45.99. Entry appears in Transaction History. | Transaction was saved successfully. Total Expenses updated to reflect the $45.99 addition. The new entry appeared at the top of Transaction History with the Food category badge. | Pass |
| Add 2 | Add a valid income transaction | User is logged in and on Dashboard. | Type: Income, Amount: 2500.00, Category: Other, Date: 2024-02-01, Notes: 'Monthly salary' | 1. Select 'Income' in Add Transaction form<br>2. Enter Amount: 2500.00<br>3. Select Category: Other<br>4. Enter Date: 2024-02-01<br>5. Click 'Add Transaction' | Transaction saved. Total Income increases by $2,500.00. Balance increases by $2,500.00. Entry visible in history with green prefix. | Income transaction was saved. Total Income and Balance both updated immediately. Entry appeared in Transaction History with a green +$2,500.00 amount. | Pass |
| Add 3 | Add transaction with zero amount | User is logged in. | Type: Expense, Amount: 0, Category: Food, | 1. Select 'Expense'<br>2. Enter Amount: 0<br>3. Select Category: Food<br>4. Click 'Add Transaction' | Validation error: 'Amount must be greater than 0.' | Validation error 'Amount must be greater than 0.' was shown | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | Date: today | | Transaction not saved. | below the Amount field. No transaction was created in the database. | |
| Add 4 | Add transaction without selecting a category | User is logged in. | Type: Expense, Amount: 20.00, Category: (not selected), Date: today | 1. Select 'Expense'<br>2. Enter Amount: 20.00<br>3. Leave Category unselected<br>4. Click 'Add Transaction' | Validation error: 'Category is required.' Transaction not submitted. | Validation error 'Category is required.' appeared below the Category dropdown. Form was not submitted and no transaction was stored. | Pass |
| Add 5 | Edit an existing transaction | User has at least one saved transaction. | Updated Amount: 50.00, Updated Category: Rent | 1. Click 'Edit' on an existing transaction in history<br>2. Update Amount to 50.00<br>3. Update Category to Rent<br>4. Click Save (PUT /api/transactions/:id) | Transaction updated in database. History reflects new values. Balance and totals recalculated correctly. | Transaction was updated via PUT request. Transaction History displayed the new amount and Rent category. Balance and totals recalculated correctly. | Pass |

## 6.4 Use Case: Filter Transactions

| **Test Case Id** | **Test case description** | **Precondition** | **Test Data** | **Test Steps** | **Expected Results** | **Actual Results** | **Status (Pass/Fail)** |
|---|---|---|---|---|---|---|---|
| Filter 1 | Filter by transaction type: | User has both income and | Filter: Type = 'Expense' | 1. In Transaction History, open | Only expense transactions are displayed. | Transaction list updated immediately | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | Expense only | expense transactions. | | Type dropdown<br>2. Select 'Expense' | Income entries are hidden. Dropdown reflects selected filter. | to show only expense entries. Income transactions were hidden. Dropdown showed Expense as the selected value. | |
| Filter 2 | Filter by category: Food | User has transactions in multiple categories. | Filter: Category = 'Food' | 1. Open Category dropdown<br>2. Select 'Food' | Only transactions with Category equal to Food are displayed. Other category items are hidden. | Only Food category transactions appeared in the list. All other category entries were filtered out correctly. | Pass |
| Filter 3 | Filter by date range | User has transactions across multiple months. | Start Date: 2024-01-01, End Date: 2024-01-31 | 1. Enter Start Date: 2024-01-01<br>2. Enter End Date: 2024-01-31<br>3. Apply date filter | Only transactions within January 2024 are shown. Transactions outside the range are excluded. | Only transactions dated within January 2024 were displayed. Transactions from other months were excluded from the results. | Pass |
| Filter 4 | Combine type and category filters | User has various transaction types and categories. | Type: Expense, Category: Rent | 1. Set Type filter to 'Expense'<br>2. Set Category filter to 'Rent' | Only Expense transactions in the Rent category are shown. Combined filter narrows results correctly. | The combined filter correctly returned only Rent-category expense transactions. All other type and category | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | combinations were hidden. | |
| Filter 5 | Reset all filters to show all transactions | Active filters are applied (Type = Expense, Category = Food). | N/A | 1. Reset Type dropdown to 'All Types'<br>2. Reset Category dropdown to 'All Categories' | All transactions (income and expense, all categories) are displayed. Dashboard returns to default unfiltered view. | All transactions were restored in the list after resetting both dropdowns. Dashboard returned to the full unfiltered view with all income and expense entries visible. | Pass |

## 6.5 Use Case: Delete Transaction

| **Test Case Id** | **Test case description** | **Precondition** | **Test Data** | **Test Steps** | **Expected Results** | **Actual Results** | **Status (Pass/Fail)** |
|---|---|---|---|---|---|---|---|
| Del 1 | Delete an expense transaction | User is logged in with at least one expense transaction. | Expense transaction: Food, $45.99 | 1. In Transaction History, locate the $45.99 Food expense<br>2. Click 'Delete' button<br>3. Confirm deletion | Transaction removed from database. History no longer shows the entry. Total Expenses decreases, Balance increases by $45.99. | Transaction was deleted from the database. The entry was removed from Transaction History. Total Expenses decreased and Balance increased by $45.99. | Pass |
| Del 2 | Delete an income | User is logged in | Income transactio | 1. Locate income transaction in history | Income transaction | Income transaction | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | transaction | with an income transaction present. | n: Monthly salary, $2,500.00 | 2. Click 'Delete'<br>3. Confirm | deleted. Total Income decreases, Balance decreases by $2,500.00. Entry removed from history. | was successfully deleted. Total Income and Balance both decreased by $2,500.00. Entry was removed from Transaction History. | |
| Del 3 | Attempt to delete a transaction belonging to another user | Two users have accounts; User A is logged in. | Transaction ID belonging to User B | 1. Send DELETE /api/transactions/:id where ID belongs to User B<br>2. Use User A's JWT token in the request | API returns 403 Forbidden. Transaction not deleted. Authorization ensures users can only delete their own records. | API returned 403 Forbidden as expected. User B's transaction was not affected. Authorization middleware correctly rejected the cross-user delete request. | Pass |
| Del 4 | Delete the last remaining transaction | User has exactly one transaction in the system. | Single transaction of any type. | 1. Click 'Delete' on the only transaction<br>2. Confirm deletion | Transaction deleted. History shows empty state. Balance returns to $0.00. Income and Expenses both show $0.00. | Final transaction was deleted. Transaction History displayed an empty state message. Balance, Total Income, and Total Expenses all correctly | Pass |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | showed $0.00. | |
| Del 5 | Attempt to delete a non-existent transaction | Authenticated user is logged in. | Invalid or non-existent transaction ID | 1. Send DELETE /api/transactions/nonexistentId with valid JWT | API returns 404 Not Found. No database changes. Frontend displays appropriate error notification. | API returned 404 Not Found. No changes were made to the database. The frontend displayed an error notification indicating the transaction could not be found. | Pass |

# Part 7: Tools and README File

## 7.1 Complete Tools List (

The following table lists all tools used across the SmartSpend development lifecycle.

| Phase | Activity | Tools Used |
|---|---|---|
| Requirements | Requirement Analysis | Microsoft word |
| Design | Architecture and UI Design | draw.io, PlantUML, Figma |
| Frontend | UI Implementation | React.js v18, Redux, Chart.js v4, Axios, Vite |
| Backend | API and Business Logic | Node.js v16+, Express.js v4.18, Mongoose v7, Nodemon |
| Database | Data Storage | MongoDB v5.0, MongoDB Atlas, MongoDB Compass |
| Security | Authentication and Data Protection | bcryptjs v2.4, jsonwebtoken v9.0, cors, dotenv |
| Testing | Quality Assurance | Jest v29, Supertest v6, Vite |
| Version Control | Source Code Management | Git, GitHub |
| Documentation | Project Documentation | Microsoft word |

## 7.2 README

SmartSpend is a MERN-based personal expense tracker. The project was developed across two directories:

- backend - built with Node.js, Express, and MongoDB
- frontend - built with React and Vite

### Project Structure

The repository was organized with a clear separation between the client and server sides. The backend handled all API routes, authentication, and database operations. The frontend was scaffolded using Vite, which served as the development server and build tool in place of the traditional Create React App setup.

### Environment Variables

Environment variables were configured for both the backend and frontend using .env files generated from the provided templates:

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

The backend .env file contained the MongoDB connection URI, JWT secret key, server port, and Node environment. The frontend .env file contained the API base URL used by Axios for all HTTP requests. Neither .env file was committed to version control.

**Running the Frontend**

The frontend was run locally using the following commands:

```
cd frontend
npm install
npm run dev
```

Vite served the React application at http://localhost:5173. The development server supported hot module replacement (HMR), allowing changes to reflect instantly in the browser without a full page reload.

**Running the Backend**

The backend was run locally using the following commands:

```
cd backend
npm install
npm run dev
```

The backend was run using the npm dev script. The API was accessible at http://localhost:5000 and connected to the MongoDB Atlas cloud database on startup.

**Deployment**

**Frontend:**

- Build command used: npm run build. Vite outputs the production bundle to the dist directory.
- The environment variable VITE_API_URL was set to the deployed backend URL.

**Backend (Render / Railway):**

- Environment variables MONGO_URI, JWT_SECRET, and NODE_ENV=production were configured in the hosting platform.
- CORS was configured to accept requests from the deployed frontend domain only.

**Database (MongoDB Atlas):**

- A free-tier MongoDB Atlas cluster was used for the production database.
- A dedicated database user with read/write access was created for the application.
- The connection string was stored securely in the backend .env file and never exposed in the codebase.